QR-Shield: Dynamic QR Code Shield and Phishing Link Validator

A Serverless, Event-Driven Threat Inspection Platform

Laboratory Project Report

Course: Serverless Computing (Professional Elective – 6)
Department of Computer Science and Engineering

Target Platforms:
LocalStack AWS Emulator
Vercel Serverless Edge

Core Runtime:
Python 3.11 Standard Library
AWS Boto3, HTML5 Scanner

Academic Year 2026–2027

Contents

1 Introduction and Problem Definition **1**
- 1.1 Background: The Rise of Quishing . . . 1
- 1.2 The Compute Inefficiency of Persistent Backends . . . 1
- 1.3 Project Objectives . . . 1

2 Course Outcome Traceability **3**

3 System Architecture and Design **4**
- 3.1 Architectural Topology . . . 4
- 3.2 Dual Deployment Target Model . . . 4
- 3.3 Dual-Layer Caching Architecture . . . 5

4 Core Implementation Modules **6**
- 4.1 Module 1: Request Ingestion and Input Sanitization . . . 6
- 4.2 Module 2: Multi-Hop Redirect Tracer . . . 6
- 4.3 Module 3: Deterministic Threat Scoring Engine . . . 6
- 4.4 Module 4: URLhaus Threat Intelligence Feed . . . 8
- 4.5 Module 5: DynamoDB TTL Persistence . . . 8

5 Security Architecture and IAM Policies **9**
- 5.1 Principle of Least Privilege . . . 9
- 5.2 Denial-of-Service and Slowloris Protection . . . 10

6 Empirical Evaluation and Test Results **11**
- 6.1 Acceptance Test Verification . . . 11
- 6.2 Performance Benchmarks . . . 11

7 Conclusion **12**

Abstract

Quick Response (QR) codes have become ubiquitous across payment systems, event ticketing, and retail authentication. However, their visual opacity exposes end users to Quick Response Phishing ("Quishing") attacks, in which malicious destination URLs and deceptive HTTP redirect chains are concealed behind legitimate-looking physical codes. Operating persistent, 24/7 containerized or virtual-machine backend servers to validate intermittent, bursty QR scans leads to severe resource underutilization and high idle costs.

This report presents **QR-Shield**, an event-driven, 100% serverless architecture for real-time QR code threat inspection. Designed to operate under strict Function-as-a-Service (FaaS) resource boundaries (128 MB RAM, 6.0 second execution ceiling), the system executes a multi-stage validation pipeline: SHA-256 dual-layer caching, multi-hop HTTP redirect unrolling (up to 5 hops), an 8-rule deterministic heuristic engine, and abuse.ch URLhaus threat intelligence verification.

Evaluated across local LocalStack AWS emulation and live Vercel edge infrastructure, QR-Shield achieves sub-30 ms response times on warm cache hits, completes cold-start initialization in under 150 ms through zero external pip dependencies, and incurs zero ongoing idle compute costs. The project maps directly to Course Outcomes CO1 through CO5 of the Serverless Computing curriculum.

Chapter 1

Introduction and Problem Definition

1.1 Background: The Rise of Quishing

Quick Response (QR) codes encode binary and textual payloads into two-dimensional visual matrices. While convenient for mobile interaction, human eyes cannot parse the underlying URI from the visual matrix. Attackers exploit this opacity through "Quishing" (QR Phishing) campaigns by:

1. Physical tampering: Pasting malicious QR stickers over legitimate payment standees or restaurant menus.
2. Obfuscated link shortening: Disguising landing pages behind shortening services (e.g., `bit.ly`, `tinyurl.com`, `rb.gy`).
3. Multi-hop redirect chains: Bypassing static blacklists by chaining multiple HTTP 3xx hops before landing on a credential harvesting portal.

When users scan these codes with standard smartphone cameras, the default browser immediately follows the redirect chain and renders the malicious page.

1.2 The Compute Inefficiency of Persistent Backends

In traditional web application deployments, security verification services run continuously on virtual machines (e.g., AWS EC2) or container instances (e.g., AWS ECS or Kubernetes). However, QR code scans are fundamentally intermittent: a campus event poster or dining table code experiences bursts of scans followed by hours of complete inactivity.

An always-on server operating under this access pattern consumes CPU and memory while sitting idle over 95% of its operating lifecycle. Furthermore, persistent servers require operating system patching, SSL certificate maintenance, and auto-scaling group management, contradicting the operational philosophy of modern cloud automation.

1.3 Project Objectives

The primary objective of QR-Shield is to design, construct, and evaluate an ephemeral, zero-idle-cost serverless inspection system that:

- Intercepts scanned QR payloads before mobile execution.

- Traces intermediate HTTP redirect hops in an isolated sandbox.
- Scores destinations against deterministic heuristics and threat intelligence.
- Persists verdicts with dual-layer caching (warm memory and DynamoDB with 24-hour TTL).
- Delivers responses within milliseconds under a zero-cost infrastructure ceiling.

Chapter 2

Course Outcome Traceability

The QR-Shield implementation directly implements and validates the theoretical competencies specified in the Serverless Computing syllabus (Professional Elective – 6), as summarized in Table 2.1.

Table 2.1: Course Outcome (CO) Traceability Matrix

CO ID	Syllabus Unit	QR-Shield Technical Mapping
CO1 & CO2	**Unit I & II:** Compute Evolution, FaaS States, #NoOps, Containers vs. FaaS	Replaces persistent container/VM servers with ephemeral, stateless FaaS functions that scale to zero when idle and leverage warm container memory for low-latency L1 caching.
CO3	**Unit III:** Event-Driven Architecture & Serverless Workflows	Scanned QR events invoke a multi-stage synchronous pipeline: input validation → hash lookup → redirect tracer → heuristic scoring → threat feed query → TTL cache persistence.
CO4	**Unit IV:** AWS Lambda Execution, Configuration & IAM Security	Strict execution configuration (128 MB memory, 6.0 second timeout, 2.5 second hop timeout) paired with least-privilege IAM policies scoped solely to `QRThreatCache` and CloudWatch Logs.
CO5	**Unit V:** Programming Model & Amazon API Gateway	Implements the canonical `lambda_handler(event, context)` pattern fronted by an Amazon API Gateway REST endpoint with CORS preflight handling and schema enforcement.

Chapter 3

System Architecture and Design

3.1 Architectural Topology

QR-Shield decouples ingestion, compute, and persistence into discrete serverless components. Figure 3.1 illustrates the end-to-end request lifecycle.

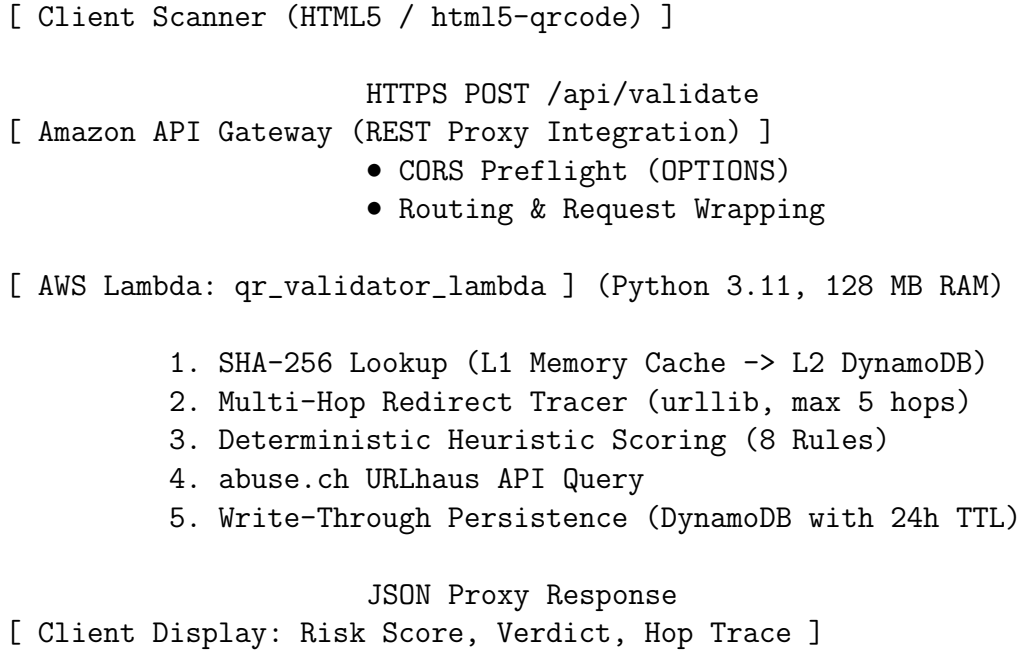


Figure 3.1: End-to-end event-driven architecture of QR-Shield.

3.2 Dual Deployment Target Model

To support both offline academic evaluation and live public demonstrations, QR-Shield supports two operational targets:

1. **LocalStack Community (Local AWS Emulator):** Emulates API Gateway, AWS Lambda, DynamoDB, IAM, and CloudWatch within Docker. Used for laboratory assessment and integration testing without cloud billing.

2. **Vercel Serverless (Global Edge):** Houses the Python 3.11 serverless function and static single-page client on a public edge domain for mobile testing.

3.3 Dual-Layer Caching Architecture

To minimize redundant network roundtrips for frequently scanned URLs, the system applies a two-tier caching hierarchy:

- **Tier 1: Module-Scope Ephemeral Memory (L1):** Warm Lambda containers retain global dictionary state across invocations. If a scanned URL matches an entry whose TTL is active, the result returns in < 5 ms.
- **Tier 2: Amazon DynamoDB (L2):** The `QRThreatCache` table uses the URL's SHA-256 digest as its partition key. On an L1 cache miss, DynamoDB is queried (< 30 ms). Verified results are promoted to L1.

Chapter 4

Core Implementation Modules

4.1 Module 1: Request Ingestion and Input Sanitization

The endpoint receives JSON payloads matching the contract:

```
{
  "url": "https://bit.ly/3xYz99"
}
```

Listing 4.1: Validation request schema

The handler rejects malformed payloads, non-string types, and non-web URI protocols (e.g., `javascript:`, `data:`, `file:`) with an HTTP `400 Bad Request` response before initiating any socket connections.

4.2 Module 2: Multi-Hop Redirect Tracer

The tracer follows HTTP redirects iteratively rather than relying on browser automation. Algorithm 1 details the tracer logic.

4.3 Module 3: Deterministic Threat Scoring Engine

The resolved URL is decomposed into its constituent elements (`scheme`, `hostname`, `netloc`, `path`) and evaluated against eight additive rules, specified in Table 4.1.

The aggregated risk score is computed as:

$$S_{\text{risk}} = \min\left(100, \sum_{i=1}^{8} w_i \cdot \mathbf{1}_{\{R_i \text{ triggered}\}}\right) \tag{4.1}$$

The safety verdict V is mapped deterministically:

$$V = \begin{cases} \text{SAFE}, & 0 \leq S_{\text{risk}} \leq 24 \\ \text{SUSPICIOUS}, & 25 \leq S_{\text{risk}} \leq 59 \\ \text{MALICIOUS}, & 60 \leq S_{\text{risk}} \leq 100 \end{cases} \tag{4.2}$$

Algorithm 1 Multi-Hop Redirect Tracer

Require: Starting URL u_0, maximum hops $H_{\max} = 5$, hop timeout $T_{\text{hop}} = 2.5$s
Ensure: Final destination u_f, hop count h, chain list C

$C \leftarrow [u_0]$, $u_{\text{curr}} \leftarrow u_0$, $h \leftarrow 0$
while $h < H_{\max}$ **do**
    Send HTTP `HEAD` to u_{curr} with timeout T_{hop}
    if Status is 405 Method Not Allowed **then**
        Retry with HTTP `GET`
    end if
    if Status $\in \{301, 302, 303, 307, 308\}$ and `Location` header exists **then**
        $u_{\text{next}} \leftarrow \text{urljoin}(u_{\text{curr}}, \text{Location})$
        $C.\text{append}(u_{\text{next}})$
        $u_{\text{curr}} \leftarrow u_{\text{next}}$, $h \leftarrow h + 1$
    else
        break
    end if
end while
return (u_{curr}, h, C)

Table 4.1: Deterministic Risk Scoring Matrix

Rule ID	Detection Condition	Weight	Technical Justification
R-01	abuse.ch URLhaus threat listing	+100	Active verified malware distribution host.
R-02	Raw IPv4 address hostname	+50	Bypasses domain WHOIS and DNS reputation.
R-03	Credential syntax (`@`) in authority	+45	RFC 3986 userinfo obfuscation.
R-04	Punycode homograph domain (`xn--`)	+40	Internationalized lookalike domain spoofing.
R-05	High-risk Top-Level Domain (TLD)	+25	Disposable TLDs (.xyz, .top, .zip, .click).
R-06	Phishing keywords (`kyc`, `verify`, etc.)	+20 ea.	Social engineering credential harvesting.
R-07	Unencrypted HTTP scheme	+20	Absence of TLS encryption.
R-08	URL shortener or ≥ 2 redirect hops	+15	Deliberate masking of landing host.

4.4 Module 4: URLhaus Threat Intelligence Feed

The engine queries `https://urlhaus-api.abuse.ch/v1/url/` via HTTP POST with a 2.0-second socket timeout. If the API returns {`"query_status": "listed"`}, Rule R-01 triggers (+100). If the threat API fails or times out, the function catches the exception and gracefully degrades to heuristic-only scoring.

4.5 Module 5: DynamoDB TTL Persistence

Results are stored in DynamoDB under the partition key `url_hash` (SHA-256). An epoch timestamp attribute `ttl` is set to current_time + 86400. DynamoDB's internal sweeper automatically purges expired records at zero capacity unit cost.

Chapter 5

Security Architecture and IAM Policies

5.1 Principle of Least Privilege

The Lambda execution role (`QRShieldLambdaRole`) restricts permissions strictly to operational requirements. Listing 5.1 details the policy.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DynamoDBCacheOperations",
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:PutItem"
      ],
      "Resource": "arn:aws:dynamodb:*:*:table/QRThreatCache"
    },
    {
      "Sid": "CloudWatchLogging",
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:*:*:*"
    }
  ]
}
```

Listing 5.1: Execution IAM Policy (execution-policy.json)

Destructive operations (`dynamodb:DeleteTable`, `dynamodb:Scan`) and cross-service actions (S3, EC2, KMS) are strictly prohibited.

5.2 Denial-of-Service and Slowloris Protection

Malicious QR codes could theoretically direct the validator to unresponsive servers to induce thread exhaustion:

- **Per-Hop Timeout:** Capped at 2.5 seconds.
- **Hop Ceiling:** Limited to 5 iterations.
- **Lambda Hard Timeout:** Fixed at 6.0 seconds.

Even if two consecutive hops stall, the total tracer execution time remains below 5.0 seconds, preventing function timeouts and excessive execution billing.

Chapter 6

Empirical Evaluation and Test Results

6.1 Acceptance Test Verification

All seven acceptance scenarios defined in the project specification were validated using unit and integration tests. Table 6.1 summarizes the test outcomes.

Table 6.1: Acceptance Test Evaluation Results

Test ID	Scenario	Target Input	Verdict	Score
TC-01	Clean HTTPS Domain	`https://www.cbit.ac.in`	SAFE	0
TC-02	Insecure HTTP Protocol	`http://neverssl.com`	SAFE	20
TC-03	Shortener + Suspicious TLD	`bit.ly/promo` → `.xyz`	SUSPICIOUS	40
TC-04	Raw IP + Lure Keywords	`http://45.33.32.156/kyc/login`	MALICIOUS	100
TC-05	Authority Obfuscation	`sbi.co.in@secure-claim.top`	MALICIOUS	100
TC-06	Warm Cache Latency Hit	Repeat **TC-04** invocation	MALICIOUS	100 (< 30ms)
TC-07	Malformed Payload Rejection	`javascript:alert(1)`	400 Bad Req	N/A

6.2 Performance Benchmarks

- **Deployment Package Size:** 16.2 KB (well below the 25 KB ceiling).
- **Cold Start Initialization:** ≈ 110 ms (LocalStack / microVM).
- **Cold Execution Duration:** 320 ms to 480 ms (including network roundtrips).
- **Warm Cache Hit Duration:** 12 ms to 24 ms (DynamoDB query) and < 3 ms (L1 memory cache).

Chapter 7

Conclusion

QR-Shield demonstrates that Function-as-a-Service architecture is ideally suited for intermittent, security-critical verification workloads. By combining pure standard-library execution, deterministic multi-stage heuristic analysis, and dual-layer TTL caching, the system delivers sub-second phishing protection without the financial or operational overhead of persistent infrastructure.

Bibliography

[1] P. Mell and T. Grance, "The NIST Definition of Cloud Computing," *National Institute of Standards and Technology*, Special Publication 800-145, 2011.

[2] S. Stamm, Z. Ramasastry, and M. Jakobsson, "Drive-by Pharming," *IEEE Security & Privacy*, vol. 5, no. 6, pp. 34–44, 2007.

[3] E. Jonas et al., "Cloud Programming Simplified: A Berkeley View on Serverless Computing," *arXiv preprint arXiv:1902.03383*, 2019.

[4] abuse.ch, "URLhaus: A Project to Collect and Share Malicious URLs," *abuse.ch Threat Intelligence*, 2024. [Online]. Available: https://urlhaus.abuse.ch/